

# DOCUMENTATION D'INSTALLATION

&

## CONFIGURATION DE SURICATA SUR WAZUH



AMÉLIE GARNIER

---

## **Table des matières :**

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Introduction</b>                                    | <b>5</b>  |
| Contexte et objectifs                                  | 5         |
| Enjeux de sécurité                                     | 5         |
| Périmètre du projet                                    | 5         |
| Architecture globale du système                        | 6         |
| <b>1. Configuration réseau du système</b>              | <b>8</b>  |
| <b>2. Installation de Suricata</b>                     | <b>9</b>  |
| 2.1 Mise à jour du système                             | 9         |
| 2.2 Installation des dépendances                       | 9         |
| <b>3. Configuration de Suricata</b>                    | <b>12</b> |
| 3.1 Configuration de la capture de paquets (AF_PACKET) | 12        |
| 3.2 Définition des zones réseau                        | 14        |
| 3.3 Configuration des ports de services                | 15        |
| 3.4 Configuration des logs EVE-JSON                    | 15        |
| <b>4. Gestion des sources de règles de détection</b>   | <b>17</b> |
| 4.1 Activation de la source Emerging Threats Open      | 17        |
| 4.2 Sources de règles disponibles                      | 18        |
| <b>5. Déploiement et activation des règles</b>         | <b>19</b> |
| 5.1 Processus de mise à jour initiale                  | 19        |
| 5.2 Statistiques de déploiement des règles             | 21        |
| 5.3 Validation de la configuration                     | 22        |
| <b>6. Synthèse de l'installation</b>                   | <b>22</b> |
| 6.1 Composants logiciels déployés                      | 22        |
| 6.2 Architecture des fichiers de configuration         | 23        |
| 6.3 État opérationnel du système                       | 23        |
| <b>7. Intégration de Suricata avec wazuh</b>           | <b>24</b> |
| 7.1 Configuration du fichier ossec.conf                | 24        |
| 7.2 Résolution des problèmes de décodage JSON          | 24        |
| 7.3 Correction des permissions et redémarrage          | 26        |
| 7.4 Vérification de l'intégration                      | 27        |
| <b>8. Tests de détection et validation</b>             | <b>27</b> |
| 8.1 Génération d'événements de test                    | 27        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| 8.2 Détection des alertes dans Wazuh                     | 27        |
| 8.3 Analyse détaillée de l'alerte                        | 29        |
| <b>9. Configuration et démarrage du service Suricata</b> | <b>30</b> |
| 9.1 Activation du démarrage automatique                  | 30        |
| 9.2 Démarrage et vérification du service                 | 30        |
| 9.3 Automatisation des mises à jour des règles           | 31        |
| 9.4 Configuration de la rotation des logs                | 32        |
| <b>10. Vérification et analyse des règles</b>            | <b>33</b> |
| 10.1 Inspection du répertoire des règles                 | 33        |
| <b>Conclusion</b>                                        | <b>34</b> |
| Synthèse technique                                       | 34        |
| Points clés de l'implémentation                          | 34        |
| Intégration Wazuh                                        | 35        |
| Tests et validation                                      | 35        |
| Performances système                                     | 35        |
| <b>Recommandations et prochaines étapes</b>              | <b>36</b> |
| Personnalisation des règles                              | 36        |
| Configuration des alertes                                | 36        |
| Documentation des procédures                             | 36        |
| Tests de charge                                          | 36        |
| Supervision continue                                     | 36        |
| Formation et sensibilisation                             | 37        |
| <b>État final du système</b>                             | <b>37</b> |
| Conformité et bonnes pratiques                           | 38        |
| Support et maintenance                                   | 38        |

## **Table des illustrations :**

|                                                                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Screenshot 1 - Architecture globale du système                                                                                                                 | 6  |
| Screenshot 2 - Configuration de Suricata                                                                                                                       | 7  |
| Screenshot 3 - Affichage de la configuration réseau avec <code>ip addr show</code>                                                                             | 8  |
| Screenshot 4 - Mise à jour complète du système Debian avec <code>apt update &amp;&amp; apt upgrade -y</code>                                                   | 9  |
| Screenshot 5 - Installation de Suricata 6.0.10 et jq 1.6-2.1 avec <code>apt install -y suricata jq</code>                                                      | 10 |
| Screenshot 6 - Installation du gestionnaire de dépôts avec <code>apt install -y software-properties-common</code>                                              | 10 |
| Screenshot 7 - Installation de pip et 13 dépendances Python avec <code>apt install -y python3-pip</code>                                                       | 11 |
| Screenshot 8 - Édition de la configuration principale avec <code>micro /etc/suricata/suricata.yaml</code>                                                      | 12 |
| Screenshot 9 - Configuration AF_PACKET (ens192, 2 threads, cluster_flow) - <code>suricata.yaml</code> lignes 587-623                                           | 13 |
| Screenshot 10 - Définition des zones réseau (HOME_NET, EXTERNAL_NET) - <code>suricata.yaml</code> lignes 15-51                                                 | 14 |
| Screenshot 11 - Configuration EVE-JSON (eve.json) - <code>suricata.yaml</code> lignes 85-118                                                                   | 16 |
| Screenshot 12 - Examen du fichier eve.json avec <code>tail -f /var/log/suricata/eve.json</code>                                                                | 17 |
| Screenshot 13 - Activation Emerging Threats Open avec <code>suricata-update enable-source et/open</code>                                                       | 17 |
| Screenshot 14 - Liste des sources disponibles (et/open, et/pro, oisf/trafficid) avec <code>suricata-update list-sources</code>                                 | 18 |
| Screenshot 15 - Téléchargement des règles (5,2 Mo depuis <a href="https://www.emergingthreats.net">emergingthreats.net</a> ) avec <code>suricata-update</code> | 20 |
| Screenshot 16 - Déploiement de 46334 règles actives sur 62149 chargées avec <code>suricata-update</code> (suite)                                               | 21 |
| Screenshot 17 - Validation de la configuration avec <code>suricata -T -c /etc/suricata/suricata.yaml -v</code>                                                 | 22 |
| Screenshot 18 - Édition de la configuration Wazuh avec <code>micro /var/ossec/etc/ossec.conf</code>                                                            | 24 |
| Screenshot 19 - Configuration localfile (eve.json, format JSON) - <code>ossec.conf</code> lignes 346-351                                                       | 24 |
| Screenshot 20 - Événements "stats" volumineux causant les erreurs avec <code>tail -f /var/ossec/logs/ossec.log</code>                                          | 24 |
| Screenshot 21 - Édition du fichier <code>internal_options.conf</code> avec <code>micro /var/ossec/etc/internal_options.conf</code>                             | 25 |
| Screenshot 22 - Configuration <code>analysisd.decoder_order_size=512</code> - <code>internal_options.conf</code> ligne 30                                      | 25 |
| Screenshot 23 - Exemple d'événement DNS formaté JSON avec <code>tail -50 /var/log/suricata/eve.json   jq</code>                                                | 26 |
| Screenshot 24 - Correction des permissions avec <code>chmod 644 &amp;&amp; chown root:root &amp;&amp; systemctl restart</code>                                 | 26 |
| Screenshot 25 - Vérification de l'intégration                                                                                                                  | 27 |
| Screenshot 26 - Génération d'événements de test                                                                                                                | 27 |
| Screenshot 27 - Visualisation des alertes dans l'interface Wazuh Threat Hunting                                                                                | 28 |
| Screenshot 28 - Métadonnées complètes de l'alerte GPL ATTACK_RESPONSE dans Document Details Wazuh                                                              | 29 |
| Screenshot 29 - Activation du service avec <code>systemctl enable suricata</code>                                                                              | 30 |
| Screenshot 30 - Démarrage et vérification du service avec <code>systemctl start suricata ; systemctl status Suricata</code>                                    | 30 |
| Screenshot 31 - Création du crontab avec sélection de l'éditeur nano                                                                                           | 31 |
| Screenshot 32 - Édition du crontab dans nano avec la tâche <code>suricata-update</code>                                                                        | 31 |
| Screenshot 33 - Vérification du crontab avec <code>crontab -l</code>                                                                                           | 32 |
| Screenshot 34 - Édition du fichier de configuration des rotations avec <code>micro /etc/logrotate.d/suricata</code>                                            | 33 |
| Screenshot 35 - Configuration logrotate pour Suricata                                                                                                          | 33 |
| Screenshot 36 - Vérification des fichiers avec <code>ls -la /var/lib/suricata/rules/</code>                                                                    | 34 |



## Introduction

### Contexte et objectifs

Ce document présente l'installation et la configuration de Suricata, un système de détection d'intrusions (IDS/IPS) open source, sur une plateforme Wazuh hébergée sur une machine virtuelle Debian 12. L'objectif est de mettre en place une solution de surveillance réseau capable d'analyser le trafic en temps réel et de détecter les activités malveillantes.

Le système est déployé en mode IDS (*Intrusion Detection System*), assurant une surveillance passive du trafic sans blocage. Une évolution vers le mode IPS (*Intrusion Prevention System*) avec blocage actif via NFQUEUE est possible ultérieurement.

### Enjeux de sécurité

L'intégration de Suricata avec Wazuh renforce la posture de sécurité en offrant une visibilité complète sur le trafic réseau. Cette solution détecte les tentatives d'intrusion, identifie les anomalies de comportement et les communications avec des serveurs malveillants. La combinaison *Suricata-Wazuh* offre une défense en profondeur efficace.

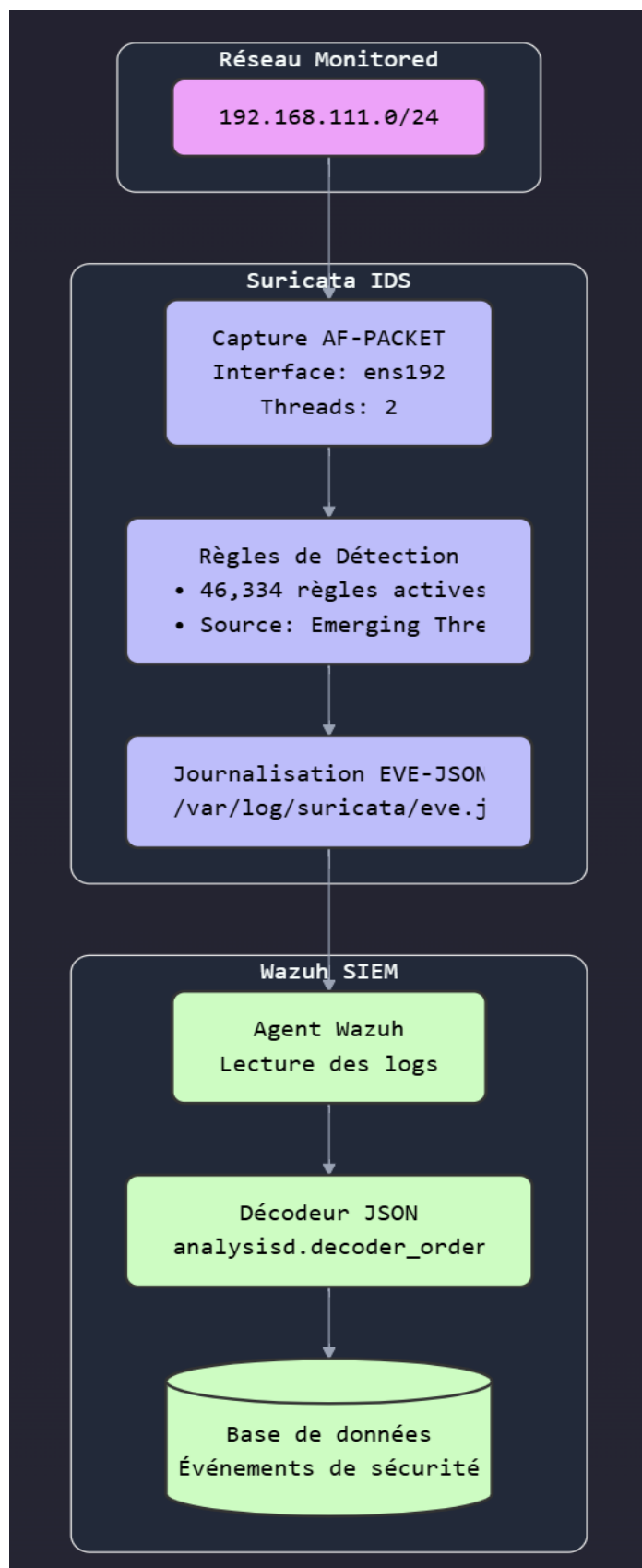
### Périmètre du projet

Cette installation concerne une machine virtuelle Debian 12 sur le réseau 192.168.111.0/24, surveillant le trafic transitant par l'interface ens192 avec les règles *Emerging Threats*.



## Architecture globale du système

Screenshot 1 - Architecture globale du système



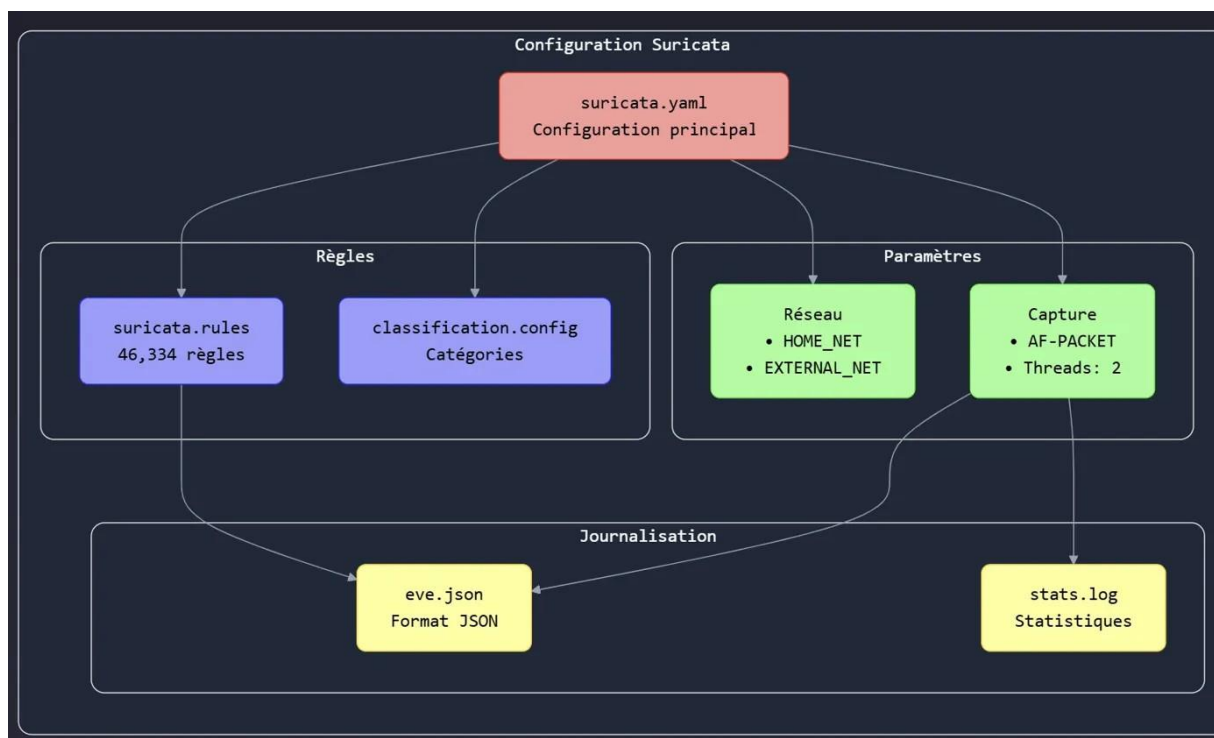
## Flux de détection :

Réseau → Suricata → Wazuh

1. **Capture** : Trafic réseau intercepté (ens192)
2. **Analyse** : 46 334 règles détectent les menaces
3. **Logs** : Événements JSON (eve.json)
4. **SIEM** : Alertes centralisées et visibles

Trafic → Détection → Alerte

Screenshot 2 - Configuration de Suricata



## Ce schéma détaille l'organisation des composants :

- **Configuration principale** : `suricata.yaml`
- **Règles** : `suricata.rules` (46 334 règles) et `classification.config`
- **Paramètres réseau** : HOME\_NET, EXTERNAL\_NET
- **Capture** : AF-PACKET avec 2 threads
- **Journalisation** : `eve.json` et `stats.log`

## 1. Configuration réseau du système

*Screenshot 3 - Affichage de la configuration réseau avec ip addr show*

```
root@debian-wazu:/home/debian-wazu# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens192: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:b2:a7:a9 brd ff:ff:ff:ff:ff:ff
    altname enp11s0
    inet 192.168.111.62/24 brd 192.168.111.255 scope global noprefixroute ens192
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feb2:a7a9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
root@debian-wazu:/home/debian-wazu#
```

Le système dispose de deux interfaces réseau opérationnelles. L'interface loopback (lo) est configurée avec l'adresse standard *127.0.0.1* pour les communications internes du système. L'interface principale *ens192*, qui sera utilisée pour la surveillance du trafic, est configurée avec l'adresse IP *192.168.111.0* sur un sous-réseau */24*.

### Configuration détaillée :

- **Interface :** *ens192*
- **État :** UP (opérationnelle)
- **Type :** Ethernet
- **Adresse IPv4 :** *192.168.111.62/24*
- **Adresse MAC :** *00:0c:29:b2:a7:a9*
- **Broadcast :** *192.168.111.255*
- **MTU :** 1500 octets

Cette configuration réseau constitue la base pour la surveillance du trafic par Suricata. L'adresse IP permet d'identifier le système sur le réseau local, tandis que l'adresse MAC assure l'identification unique au niveau de la couche liaison de données.





## 2. Installation de Suricata

### 2.1 Mise à jour du système

Avant toute installation de nouveaux composants, une mise à jour complète du système Debian est nécessaire pour garantir la disponibilité des derniers correctifs de sécurité et la compatibilité des paquets.

#### **Commandes exécutées :**

*Screenshot 4 - Mise à jour complète du système Debian avec apt update && apt upgrade -y*

```
root@debian-wazu:/home/debian-wazu# apt update && apt upgrade -y
Atteint :1 https://packages.wazuh.com/4.x/apt stable InRelease
Atteint :2 http://deb.debian.org/debian bookworm InRelease
Atteint :3 http://security.debian.org/debian-security bookworm-security InRelease
Réception de :4 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
55,4 ko réceptionnés en 2s (36,3 ko/s)
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
7 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Calcul de la mise à jour... Fait
Les NOUVEAUX paquets suivants seront installés :
  linux-image-6.1.0-41-amd64
Les paquets suivants seront mis à jour :
  firefox-esr firefox-esr-l10n-fr linux-image-amd64 linux-libc-dev wazuh-dashboard wazuh-indexer wazuh-manager
7 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 0 o/1 652 Mo dans les archives.
Après cette opération, 416 Mo d'espace disque supplémentaires seront utilisés.
```

**Résultat** → Tous les paquets sont à jour, le système est prêt pour l'installation de Suricata.

### 2.2 Installation des dépendances

L'installation de Suricata nécessite plusieurs composants essentiels pour son fonctionnement optimal et son intégration avec Wazuh.



Screenshot 5 - Installation de Suricata 6.0.10 et jq 1.6-2.1 avec `apt install -y suricata jq`

```

root@debian-wazu:/home/debian-wazu# apt install -y suricata jq
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
suricata est déjà la version la plus récente (1:6.0.10-1).
jq est déjà la version la plus récente (1.6-2.1+deb12u1).
jq passé en « installé manuellement ».
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  linux-image-6.1.0-25-amd64
Veuillez utiliser « apt autoremove » pour le supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@debian-wazu:/home/debian-wazu#

```

Installation de *Suricata version 6.0.10*, le moteur de détection d'intrusions, et de *jq version 1.6-2.1*, *utilitaire* indispensable pour le traitement et l'analyse des fichiers JSON générés par Suricata.

Screenshot 6 - Installation du gestionnaire de dépôts avec `apt install -y software-properties-common`

```

root@debian-wazu:/home/debian-wazu# apt install -y software-properties-common
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
software-properties-common est déjà la version la plus récente (0.99.30-4.1~deb12u1).
software-properties-common passé en « installé manuellement ».
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  linux-image-6.1.0-25-amd64
Veuillez utiliser « apt autoremove » pour le supprimer.
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@debian-wazu:/home/debian-wazu#

```

Installation de *software-properties-common version 0.99.30-4.1*, permettant la gestion avancée des dépôts de logiciels et facilitant l'ajout de sources tierces si nécessaire.



Screenshot 7 - Installation de pip et 13 dépendances Python avec apt install -y python3-pip

```

root@debian-wazu:/home/debian-wazu# apt install -y python3-pip
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  linux-image-6.1.0-25-amd64
Veuillez utiliser « apt autoremove » pour le supprimer.
Les paquets supplémentaires suivants seront installés :
  libexpat1-dev libjs-jquery libjs-sphinxdoc libjs-underscore libpython3-dev libpython3.11-dev
  python3-dev python3-distutils python3-lib2to3 python3-setuptools python3-wheel python3.11-dev
Paquets suggérés :
  python-setuptools-doc
Les NOUVEAUX paquets suivants seront installés :
  libexpat1-dev libjs-jquery libjs-sphinxdoc libjs-underscore libpython3-dev libpython3.11-dev
  python3-dev python3-distutils python3-lib2to3 python3-pip python3-setuptools python3-wheel
  python3.11-dev
0 mis à jour, 13 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 8 201 ko dans les archives.
Après cette opération, 39,9 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 http://deb.debian.org/debian bookworm/main amd64 libexpat1-dev amd64 2.5.0-1+deb12u2
[151 kB]
Réception de :2 http://deb.debian.org/debian bookworm/main amd64 libjs-jquery all 3.6.1+dfsg+~3.5.14-
1 [326 kB]
Réception de :3 http://deb.debian.org/debian bookworm/main amd64 libjs-underscore all 1.13.4~dfsg+~1.

```

Installation de python3-pip et de ses dépendances pour permettre l'installation de modules Python nécessaires à l'intégration avec l'écosystème Wazuh et d'autres outils d'analyse de sécurité.

## Packages installés :

- **Suricata** → 6.0.10
- **jq** → 1.6-2.1
- **software-properties-common** → 0.99.30-4.1
- **python3-pip** → Dernière version disponible

## Dépendance automatique :

Le système a automatiquement installé le paquet *linux-image-6.1.0-25-amd64* comme dépendance, assurant la compatibilité du noyau avec les fonctionnalités avancées de capture réseau *AF\_PACKET* utilisées par Suricata.



### 3. Configuration de Suricata

#### 3.1 Configuration de la capture de paquets (AF\_PACKET)

La configuration du mode de capture *AF\_PACKET* est essentielle pour assurer des performances optimales de Suricata. Ce mode utilise les fonctionnalités avancées du noyau Linux pour capturer les paquets réseau avec une efficacité maximale.

##### Édition du fichier de configuration principal :

*Screenshot 8 - Édition de la configuration principale avec micro /etc/suricata/suricata.yaml*

```
root@debian-wazu:/home/debian-wazu# micro /etc/suricata/suricata.yaml
```

##### Configuration AF\_PACKET dans /etc/suricata/suricata.yaml :

```
af-packet:  
- interface: ens192  
  
  threads: 2  
  
  cluster-id: 99  
  
  cluster-type: cluster_flow  
  
  defrag: yes  
  
  use-mmap: yes
```



Screenshot 9 - Configuration AF\_PACKET (ens192, 2 threads, cluster\_flow) - suricata.yaml lignes 587-623

```

587 # Linux high speed capture support
588 af-packet:
589   - interface: ens192
590     threads: 2
591     # Number of receive threads. "auto" uses the number of cores
592     #threads: auto
593     # Default clusterid. AF_PACKET will load balance packets based on flow.
594     cluster-id: 99
595     # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
596     # This is only supported for Linux kernel > 3.1
597     # possible value are:
598     # * cluster_flow: all packets of a given flow are sent to the same socket
599     # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
600     # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
601     # socket. Requires at least Linux 3.14.
602     # * cluster_ebpf: eBPF file load balancing. See doc/userguide/capture-hardware/ebpf-xdp.rst for
603     # more info.
604     # Recommended modes are cluster_flow on most boxes and cluster_cpu or cluster_qm on system
605     # with capture card using RSS (requires cpu affinity tuning and system IRQ tuning)
606     cluster-type: cluster_flow
607     # In some fragmentation cases, the hash can not be computed. If "defrag" is set
608     # to yes, the kernel will do the needed defragmentation before sending the packets.
609     defrag: yes
610     use-mmap: yes
611     # To use the ring feature of AF_PACKET, set 'use-mmap' to yes
612     #use-mmap: yes
613     # Lock memory map to avoid it being swapped. Be careful that over
614     # subscribing could lock your system
615     #mmap-locked: yes
616     # Use tpacket_v3 capture mode, only active if use-mmap is true
617     # Don't use it in IPS or TAP mode as it causes severe latency
618     #tpacket-v3: yes
619     # Ring size will be computed with respect to "max-pending-packets" and number
620     # of threads. You can set manually the ring size in number of packets by setting
621     # the following value. If you are using flow "cluster-type" and have really network
622     # intensive single-flow you may want to set the "ring-size" independently of the number
623     # of threads:

```

## Explications détaillées :

- **interface: ens192** → Interface réseau principale du système
- **threads: 2** → Traitement parallèle adapté aux 2 cœurs CPU disponibles sur la VM. Le nombre de threads doit correspondre au nombre de cœurs pour optimiser les performances sans surcharge.
- **cluster-id: 99** → Identifiant unique pour l'équilibrage de charge
- **cluster-type: cluster\_flow** → Tous les paquets d'un flux vers le même thread → Garantit la cohérence de l'analyse
- **defrag: yes** → Reconstitution des paquets IP fragmentés
  - Permet l'analyse complète des communications
- **use-mmap: yes** → Amélioration des performances avec RSS
  - Réduction de la latence de capture

L'interface ens192 a été configurée avec 2 threads de traitement pour assurer un traitement parallèle efficace des paquets capturés. Le cluster ID 99 avec le mode cluster\_flow garantit que tous les paquets appartenant à un même flux réseau sont traités par le même thread, ce qui est essentiel pour maintenir la cohérence de l'analyse et éviter les faux positifs liés à la réorganisation des paquets.





### 3.2 Définition des zones réseau

La définition précise des zones réseau permet à Suricata de distinguer efficacement le trafic interne légitime du trafic externe potentiellement malveillant, optimisant ainsi la pertinence des alertes générées.

#### Configuration des groupes d'adresses :

Screenshot 10 - Définition des zones réseau (HOME\_NET, EXTERNAL\_NET) - suricata.yaml lignes 15-51

```

15 vars:
16 # more specific is better for alert accuracy and performance
17 address-groups:
18   HOME_NET: "[192.168.111.0/24]"
19   #HOME_NET: "[192.168.0.0/16]"
20   #HOME_NET: "[10.0.0.0/8]"
21   #HOME_NET: "[172.16.0.0/12]"
22   #HOME_NET: "any"
23
24   EXTERNAL_NET: "!$HOME_NET"
25   #EXTERNAL_NET: "any"
26
27   HTTP_SERVERS: "$HOME_NET"
28   SMTP_SERVERS: "$HOME_NET"
29   SQL_SERVERS: "$HOME_NET"
30   DNS_SERVERS: "$HOME_NET"
31   TELNET_SERVERS: "$HOME_NET"
32   AIM_SERVERS: "$EXTERNAL_NET"
33   DC_SERVERS: "$HOME_NET"
34   DNP3_SERVER: "$HOME_NET"
35   DNP3_CLIENT: "$HOME_NET"
36   MODBUS_CLIENT: "$HOME_NET"
37   MODBUS_SERVER: "$HOME_NET"
38   ENIP_CLIENT: "$HOME_NET"
39   ENIP_SERVER: "$HOME_NET"
40
41 port-groups:
42   HTTP_PORTS: "80"
43   SHELLCODE_PORTS: "180"
44   ORACLE_PORTS: 1521
45   SSH_PORTS: 22
46   DNP3_PORTS: 20000
47   MODBUS_PORTS: 502
48   FILE_DATA_PORTS: "[$HTTP_PORTS,110,143]"
49   FTP_PORTS: 21
50   GENEVE_PORTS: 6081
51   VXLAN_PORTS: 4789

```

#### Analyse de la configuration :

**HOME\_NET : 192.168.111.0/24** → Périmètre de confiance (réseau interne) → 254 adresses IP utilisables

**EXTERNAL\_NET : !\$HOME\_NET** → Tout trafic externe au réseau local → Internet et réseaux tiers

#### Serveurs positionnés dans HOME\_NET :

- ✓ **Services IT** → HTTP, SMTP, SQL, DNS, Telnet
- ✓ **Protocoles industriels** → DNP3, ENIP, Modbus
- ✓ **Services d'infrastructure** → DC (Domain Controller)

#### Serveurs positionnés dans EXTERNAL\_NET :

- ✓ **AIM (AOL Instant Messenger)**



La configuration des groupes d'adresses permet à Suricata de distinguer le trafic interne du trafic externe. Le réseau *HOME\_NET* correspond au sous-réseau 192.168.111.0/24, représentant le périmètre de confiance. *EXTERNAL\_NET* est configuré comme l'inverse du *HOME\_NET*, représentant tout trafic externe.

Les serveurs HTTP, SMTP, SQL, DNS, Telnet, ainsi que les serveurs industriels (DNP3, ENIP, Modbus) sont positionnés dans *HOME\_NET*, permettant une surveillance spécifique de ces services critiques. Seuls les serveurs AIM sont positionnés dans *EXTERNAL\_NET*.

Cette configuration permet une surveillance ciblée des services critiques positionnés dans le réseau de confiance, tout en maintenant une vigilance accrue sur les communications avec l'extérieur.

### 3.3 Configuration des ports de services

Les groupes de ports identifient rapidement les protocoles utilisés et permettent une analyse protocolaire précise.

#### **Configuration des groupes de ports :**

- Le port HTTP standard 80 surveille le trafic web non chiffré
- Les ports SSH (22), FTP (21), Oracle (1521), DNP3 (20000), Modbus (502), GENEVE (6081) et VXLAN (4789) permettent une analyse protocolaire précise
- La catégorie SHELLCODE\_PORTS surveille tous les ports excepté le 80
- Le groupe FILE\_DATA\_PORTS regroupe HTTP, POP3 (110) et IMAP (143) pour surveiller les transferts de fichiers dangereux

### 3.4 Configuration des logs EVE-JSON

Le système de journalisation *EVE* (*Extensible Event Format*) constitue la pierre angulaire de l'intégration avec Wazuh, générant des logs structurés au format *JSON* facilitant l'analyse automatisée.



Screenshot 11 - Configuration EVE-JSON (eve.json) - suricata.yaml lignes 85-118

```

85
86 # Extensible Event Format (nicknamed EVE) event log in JSON format
87 - eve-log
88   enabled: yes
89   filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
90   filename: eve.json
91   # Enable for multi-threaded eve.json output; output files are amended with
92   # with an identifier, e.g., eve.9.json
93   #threaded: false
94   #prefix: "@cee: " # prefix to prepend to each log entry
95   # the following are valid when type: syslog above
96   #identity: "suricata"
97   #facility: local5
98   #level: Info ## possible levels: Emergency, Alert, Critical,
99   #         Error, Warning, Notice, Info, Debug
100  #ethernet: no # log ethernet header in events when available
101  #redis:
102  #  server: 127.0.0.1
103  #  port: 6379
104  #  async: true ## if redis replies are read asynchronously
105  #  mode: list ## possible values: list|lpush (default), rpush, channel|publish
106  #         ## lpush and rpush are using a Redis list. "list" is an alias for lpush
107  #         ## publish is using a Redis channel. "channel" is an alias for publish
108  #  key: suricata ## key or channel to use (default to suricata)
109  # Redis pipelining set up. This will enable to only do a query every
110  # 'batch-size' events. This should lower the latency induced by network
111  # connection at the cost of some memory. There is no flushing implemented
112  # so this setting should be reserved to high traffic Suricata deployments.
113  #  pipelining:
114  #    enabled: yes ## set enable to yes to enable query pipelining
115  #    batch-size: 10 ## number of entries to keep in buffer
116
117  # Include top level metadata. Default yes.
118  #metadata: no

```

## Configuration de journalisation :

- **Format** → JSON structuré (EVE)
- **Emplacement** → /var/log/suricata/eve.json
- **Type de fichier** → Régulier (regular)
- **Rotation** → Automatique (gérée par le système)

## Avantages du format EVE-JSON :

- ✓ Traitement automatisé par Wazuh
- ✓ Recherche et filtrage facilités
- ✓ Intégration avec outils SIEM
- ✓ Analyse historique performante

Le système de journalisation *EVE* (Extensible Event Format) génère des logs structurés au format JSON dans /var/log/suricata/eve.json. Ce format est adapté à l'intégration avec Wazuh car il permet un traitement automatisé des événements de sécurité, facilitant la surveillance en temps réel et l'analyse historique.



Screenshot 12 - Examen du fichier eve.json avec tail -f /var/log/suricata/eve.json

```

root@debian-wazu: /home/debian-wazu# tail -f /var/log/suricata/eve.json
{"timestamp": "2025-11-18T22:09:10.072004+0100", "event_type": "stats", "stats": {"uptime": 30, "capture": {"kernel_packets": 19, "kernel_drops": 0, "errors": 0}, "decoder": {"pkts": 19, "bytes": 5551, "invalid": 0, "ipv4": 19, "ipv6": 0, "ethernet": 19, "chdlc": 0, "raw": 0, "null": 0, "sll": 0, "tcp": 19, "udp": 0, "sctp": 0, "icmpv4": 0, "icmpv6": 0, "ppp": 0, "pppoe": 0, "geneve": 0, "gre": 0, "vlan": 0, "vlan_qinq": 0, "vxlan": 0, "vntag": 0, "ieee8021ah": 0, "teredo": 0, "ipv4_in_ipv6": 0, "ipv6_in_ipv6": 0, "mpls": 0, "avg_pkt_size": 292, "max_pkt_size": 1514, "max_mac_addr_src": 0, "max_mac_addr_dst": 0, "erspan": 0, "event": {"ipv4": {"pkt_too_small": 0, "hlen_too_small": 0, "iplen_smaller_than_hlen": 0, "trunc_pkt": 0, "opt_invalid": 0, "opt_invalid_len": 0, "opt_malformed": 0, "opt_pad_required": 0, "opt_eol_required": 0, "opt_duplicate": 0, "opt_unknown": 0, "wrong_ip_version": 0, "icmpv6": 0, "frag_pkt_too_large": 0, "frag_overlap": 0, "frag_ignored": 0}, "icmpv4": {"pkt_too_small": 0, "unknown_type": 0, "unknown_code": 0, "ipv4_trunc_pkt": 0, "ipv4_unknown_ver": 0}, "icmpv6": {"unknown_type": 0, "unknown_code": 0, "pkt_too_small": 0, "ipv6_unknown_version": 0, "ipv6_trunc_pkt": 0, "mld_message_with_invalid_hl": 0, "unassigned_type": 0, "experimentation_type": 0}, "ipv6": {"pkt_too_small": 0, "trunc_pkt": 0, "trunc_exthdr": 0, "exthdr_dupl_fh": 0, "exthdr_useless_fh": 0, "exthdr_dupl_rh": 0, "exthdr_dupl_hh": 0, "exthdr_dupl_ah": 0, "exthdr_dupl_eh": 0, "exthdr_invalid_optlen": 0, "wrong_ip_version": 0, "exthdr_ah_res_not_null": 0, "hopopts_unknown_opt": 0, "hopopts_only_padding": 0, "dstopts_unknown_opt": 0, "dstopts_only_padding": 0, "rh_type_0": 0, "zero_len_padn": 0, "fh_non_zero_reserved_field": 0, "data_after_none_header": 0, "unknown_next_header": 0, "icmpv4": {"frag_pkt_too_large": 0, "frag_overlap": 0, "frag_invalid_length": 0, "frag_ignored": 0, "ipv4_in_ipv6_too_small": 0, "ipv4_in_ipv6_wrong_version": 0, "ipv6_in_ipv6_too_small": 0, "ipv6_in_ipv6_wrong_version": 0}, "tcp": {"pkt_too_small": 0, "hlen_too_small": 0, "invalid_optlen": 0, "opt_invalid_len": 0, "opt_duplicate": 0, "udp": {"pkt_too_small": 0, "hlen_too_small": 0, "hlen_invalid": 0, "sll": 0, "et_hernet": {"pkt_too_small": 0}, "ppp": {"pkt_too_small": 0, "vju_pkt_too_small": 0, "ip4_pkt_too_small": 0, "ip6_pkt_too_small": 0, "wrong_type": 0, "unsup_proto": 0}, "pppoe": {"pkt_too_small": 0, "wrong_code": 0, "malformed_tags": 0}, "gre": {"pkt_too_small": 0, "wrong_version": 0, "version0_recur": 0, "version0_flags": 0, "version0_hdr_too_big": 0, "version0_malformed_sre_hdr": 0, "version1_chksum": 0, "version1_route": 0, "version1_ssr": 0, "version1_recur": 0, "version1_flags": 0, "version1_no_key": 0, "version1_wrong_protocol": 0, "version1_malformed_sre_hdr": 0, "version1_hdr_too_big": 0}, "vlan": {"header_too_small": 0, "unknown_type": 0, "too_many_layers": 0}, "ieee8021ah": {"header_too_small": 0}, "vntag": {"header_too_small": 0, "unknown_type": 0}, "ipraw": {"invalid_ip_version": 0, "ltnull": {"pkt_too_small": 0, "unsupported_type": 0}, "sctp": {"pkt_too_small": 0}, "mpls": {"header_too_small": 0, "pkt_too_small": 0, "bad_label_router_alert": 0, "bad_label_implicit_null": 0, "bad_label_reserved": 0, "unknown_payload_type": 0}, "vxlan": {"unknown_payload_type": 0}, "geneve": {"unknown_payload_type": 0}, "erspan": {"header_too_small": 0, "unsupported_version": 0, "too_many_vlan_layers": 0}, "dce": {"pkt_too_small": 0}, "chdlc": {"pkt_too_small": 0}, "too_many_layers": 0, "flow": {"memcap": 0, "tcp": 3, "udp": 0, "icmpv4": 0, "icmpv6": 0, "tcp_reuse": 0, "get_used": 0, "get_used_eval": 0, "get_used_eval_reject": 0, "get_used_eval_busy": 0, "get_used_failed": 0, "wrk": {"spare_sync_avg": 100, "spare_sync": 2, "spare_sync_incomplete": 0, "spare_sync_empty": 0, "flows_evicted_needs_work": 0, "flows_evicted_pkt_inject": 0, "flows_evicted": 0, "flows_injected": 0}, "mgr": {"full_hash_pass": 1, "closed_pruned": 0, "new_pruned": 0, "est_pruned": 0, "bypassed_pruned": 0, "rows_maxlen": 0, "flows_checked": 0, "flows_notimeout": 0, "flows_timeout": 0, "flows_timeout_inuse": 0, "flows_evicted": 0, "flows_evicted_needs_work": 0}, "spare": 9900, "emerg_mode_entered": 0, "emerg_mode_over": 0, "memuse": 7474304, "defrag": {"ipv4": {"fragments": 0, "reassembled": 0, "timeouts": 0}, "ipv6": {"fragments": 0, "reassembled": 0, "timeouts": 0}, "max_frag_hits": 0, "flow_bypassed": {"local_pkts": 0, "local_bytes": 0, "local_capture_pkts": 0, "local_capture_bytes": 0, "closed": 0, "pkts": 0, "bytes": 0}, "tcp": {"sessions": 0, "ssn_memcap_drop": 0, "pseudo": 0, "pseudo_failed": 0, "invalid_checks": 0, "no_flow": 0, "syn": 0, "synack": 0, "rst": 0, "midstream_pickups": 0, "pkt_on_wrong_thread": 0, "segment_memcap_drop": 0, "stream_depth_reached": 0, "reassembly_gap": 0, "overlap": 0, "overlap_diff_data": 0, "insert_data_normal_fail": 0, "insert_data_overlap_fail": 0, "insert_list_fail": 0, "memuse": 1212416, "reassembly_memuse": 196608, "detect": {"engines": [{"id": 0, "last_reload": "2025-11-18T22:09:10.942309+0100", "rules_loaded": 46334, "rules_failed": 0}], "alert": 0, "alert_queue_overflow": 0, "alerts_suppressed": 0}, "app_layer": {"flow": {"http": 0, "ftp": 0, "smtp": 0, "tls": 0, "ssh": 0, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "nfs_tcp": 0, "ntp": 0, "ftp_data": 0, "tftp": 0, "ike_v2": 0, "krb5_tcp": 0, "dhcpc": 0, "snmp": 0, "sip": 0, "rfb": 0, "mqtt": 0, "rdp": 0, "failed_tcp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0, "failed_udp": 0}, "tx": {"http": 0, "ftp": 0, "smtp": 0, "tls": 0, "ssh": 0, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "nfs_tcp": 0, "ntp": 0, "ftp_data": 0, "tftp": 0, "ikev2": 0, "krb5_tcp": 0, "dhcpc": 0, "snmp": 0, "sip": 0, "rfb": 0, "mqtt": 0, "rdp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0}, "expectations": 0}, "http": {"memuse": 0, "memcap": 0}, "ftp": {"memuse": 0, "memcap": 0}, "file_store": {"open_files": 0}}}, {"timestamp": "2025-11-18T22:09:10.072511+0100", "event_type": "stats", "stats": {"uptime": 38, "capture": {"kernel_packets": 48, "kernel_drops": 0, "errors": 0}, "decoder": {"pkts": 48, "bytes": 20517, "invalid": 0, "ipv4": 48, "ipv6": 0, "ethernet": 48, "chdlc": 0, "raw": 0, "null": 0, "sll": 0, "tcp": 48, "udp": 0, "sctp": 0, "icmpv4": 0, "icmpv6": 0, "ppp": 0, "pppoe": 0, "geneve": 0, "gre": 0, "vlan": 0, "vlan_qinq": 0, "vxlan": 0, "vntag": 0, "ieee8021ah": 0, "teredo": 0, "ipv4_in_ipv6": 0, "ipv6_in_ipv6": 0, "mpls": 0, "avg_pkt_size": 426.8125, "max_pkt_size": 1514, "max_mac_addr_src": 0, "max_mac_addr_dst": 0, "erspan": 0, "event": {"ipv4": {"pkt_too_small": 0, "hlen_too_small": 0, "iplen_smaller_than_hlen": 0, "trunc_pkt": 0, "opt_invalid": 0, "opt_invalid_len": 0, "opt_malformed": 0, "opt_pad_required": 0, "opt_eol_required": 0, "opt_duplicate": 0, "opt_unknown": 0, "wrong_ip_version": 0, "icmpv6": 0, "frag_pkt_too_large": 0, "frag_overlap": 0, "frag_ignored": 0}, "icmpv4": {"pkt_too_small": 0, "unknown_type": 0, "unknown_code": 0, "ipv4_trunc_pkt": 0, "ipv4_unknown_ver": 0}, "icmpv6": {"unknown_type": 0, "unknown_code": 0, "pkt_too_small": 0, "ipv6_unknown_version": 0, "ipv6_trunc_pkt": 0, "mld_message_with_invalid_hl": 0, "unassigned_type": 0, "experimentation_type": 0}, "ipv6": {"pkt_too_small": 0, "trunc_pkt": 0, "trunc_exthdr": 0, "exthdr_dupl_fh": 0, "exthdr_useless_fh": 0, "exthdr_dupl_rh": 0, "exthdr_dupl_hh": 0, "exthdr_dupl_ah": 0, "exthdr_dupl_eh": 0, "exthdr_invalid_optlen": 0, "wrong_ip_version": 0, "exthdr_ah_res_not_null": 0, "hopopts_unknown_opt": 0, "hopopts_only_padding": 0, "dstopts_unknown_opt": 0, "dstopts_only_padding": 0, "rh_type_0": 0, "zero_len_padn": 0, "fh_non_zero_reserved_field": 0, "data_after_none_header": 0, "unknown_next_header": 0, "icmpv4": {"frag_pkt_too_large": 0, "frag_overlap": 0, "frag_invalid_length": 0, "frag_ignored": 0, "ipv4_in_ipv6_too_small": 0, "ipv4_in_ipv6_wrong_version": 0, "ipv6_in_ipv6_too_small": 0, "ipv6_in_ipv6_wrong_version": 0}, "tcp": {"pkt_too_small": 0, "hlen_too_small": 0, "invalid_optlen": 0, "opt_invalid_len": 0, "opt_duplicate": 0, "udp": {"pkt_too_small": 0, "hlen_too_small": 0, "hlen_invalid": 0, "sll": 0, "et_hernet": {"pkt_too_small": 0}, "ppp": {"pkt_too_small": 0, "vju_pkt_too_small": 0, "ip4_pkt_too_small": 0, "ip6_pkt_too_small": 0, "wrong_type": 0, "unsup_proto": 0}, "pppoe": {"pkt_too_small": 0, "wrong_code": 0, "malformed_tags": 0}, "gre": {"pkt_too_small": 0, "wrong_version": 0, "version0_recur": 0, "version0_flags": 0, "version0_hdr_too_big": 0, "version0_malformed_sre_hdr": 0, "version1_chksum": 0, "version1_route": 0, "version1_ssr": 0, "version1_recur": 0, "version1_flags": 0, "version1_no_key": 0, "version1_wrong_protocol": 0, "version1_malformed_sre_hdr": 0, "version1_hdr_too_big": 0}, "vlan": {"header_too_small": 0, "unknown_type": 0, "too_many_layers": 0}, "ieee8021ah": {"header_too_small": 0}, "vntag": {"header_too_small": 0, "unknown_type": 0}, "ipraw": {"invalid_ip_version": 0, "ltnull": {"pkt_too_small": 0, "unsupported_type": 0}, "sctp": {"pkt_too_small": 0}, "mpls": {"header_too_small": 0, "pkt_too_small": 0, "bad_label_router_alert": 0, "bad_label_implicit_null": 0, "bad_label_reserved": 0, "unknown_payload_type": 0}, "vxlan": {"unknown_payload_type": 0}, "geneve": {"unknown_payload_type": 0}, "erspan": {"header_too_small": 0, "unsupported_version": 0, "too_many_vlan_layers": 0}, "dce": {"pkt_too_small": 0}, "chdlc": {"pkt_too_small": 0}, "too_many_layers": 0, "flow": {"memcap": 0, "tcp": 3, "udp": 0, "icmpv4": 0, "icmpv6": 0, "tcp_reuse": 0, "get_used": 0, "get_used_eval": 0, "get_used_eval_reject": 0, "get_used_eval_busy": 0, "get_used_failed": 0, "wrk": {"spare_sync_avg": 100, "spare_sync": 2, "spare_sync_incomplete": 0, "spare_sync_empty": 0, "flows_evicted_needs_work": 0, "flows_evicted_pkt_inject": 0, "flows_evicted": 0, "flows_injected": 0}, "mgr": {"full_hash_pass": 1, "closed_pruned": 0, "new_pruned": 0, "est_pruned": 0, "bypassed_pruned": 0, "rows_maxlen": 0, "flows_checked": 0, "flows_notimeout": 0, "flows_timeout": 0, "flows_timeout_inuse": 0, "flows_evicted": 0, "flows_evicted_needs_work": 0}, "spare": 9900, "emerg_mode_entered": 0, "emerg_mode_over": 0, "memuse": 7474304, "defrag": {"ipv4": {"fragments": 0, "reassembled": 0, "timeouts": 0}, "ipv6": {"fragments": 0, "reassembled": 0, "timeouts": 0}, "max_frag_hits": 0, "flow_bypassed": {"local_pkts": 0, "local_bytes": 0, "local_capture_pkts": 0, "local_capture_bytes": 0, "closed": 0, "pkts": 0, "bytes": 0}, "tcp": {"sessions": 0, "ssn_memcap_drop": 0, "pseudo": 0, "pseudo_failed": 0, "invalid_checks": 0, "no_flow": 0, "syn": 0, "synack": 0, "rst": 0, "midstream_pickups": 0, "pkt_on_wrong_thread": 0, "segment_memcap_drop": 0, "stream_depth_reached": 0, "reassembly_gap": 0, "overlap": 0, "overlap_diff_data": 0, "insert_data_normal_fail": 0, "insert_data_overlap_fail": 0, "insert_list_fail": 0, "memuse": 1212416, "reassembly_memuse": 196608, "detect": {"engines": [{"id": 0, "last_reload": "2025-11-18T22:09:10.942309+0100", "rules_loaded": 46334, "rules_failed": 0}], "alert": 0, "alert_queue_overflow": 0, "alerts_suppressed": 0}, "app_layer": {"flow": {"http": 0, "ftp": 0, "smtp": 0, "tls": 0, "ssh": 0, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "nfs_tcp": 0, "ntp": 0, "ftp_data": 0, "tftp": 0, "ike_v2": 0, "krb5_tcp": 0, "dhcpc": 0, "snmp": 0, "sip": 0, "rfb": 0, "mqtt": 0, "rdp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0, "failed_udp": 0}, "tx": {"http": 0, "ftp": 0, "smtp": 0, "tls": 0, "ssh": 0, "imap": 0, "smb": 0, "dcerpc_tcp": 0, "dns_tcp": 0, "nfs_tcp": 0, "ntp": 0, "ftp_data": 0, "tftp": 0, "ikev2": 0, "krb5_tcp": 0, "dhcpc": 0, "snmp": 0, "sip": 0, "rfb": 0, "mqtt": 0, "rdp": 0, "dcerpc_udp": 0, "dns_udp": 0, "nfs_udp": 0, "krb5_udp": 0}, "expectations": 0}, "http": {"memuse": 0, "memcap": 0}, "ftp": {"memuse": 0, "memcap": 0}, "file_store": {"open_files": 0}}}}

```

L'examen du fichier eve.json montre la richesse des informations. Un événement DNS typique contient → timestamp, interface réseau, type d'événement, adresses IP source/destination, ports, protocole, détails DNS (type de requête, réponses, codes de retour).

Ce format permet une surveillance en temps réel et une analyse historique approfondie des événements de sécurité détectés par Suricata, avec une intégration transparente dans l'écosystème Wazuh.

## 4. Gestion des sources de règles de détection

### 4.1 Activation de la source Emerging Threats Open

Suricata utilise des signatures (règles) pour identifier les menaces réseau. L'activation de sources de règles fiables et maintenues est donc essentielle pour assurer une détection efficace des attaques.

Screenshot 13 - Activation Emerging Threats Open avec suricata-update enable-source et/open

```

root@debian-wazu: /home/debian-wazu# suricata-update enable-source et/open
18/11/2025 -- 21:58:21 - <Info> -- Using data-directory /var/lib/suricata.
18/11/2025 -- 21:58:21 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
18/11/2025 -- 21:58:21 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
18/11/2025 -- 21:58:21 - <Info> -- Found Suricata version 6.0.10 at /usr/bin/suricata.
18/11/2025 -- 21:58:21 - <Info> -- Creating directory /var/lib/suricata/update/sources
18/11/2025 -- 21:58:21 - <Info> -- Source et/open enabled
root@debian-wazu: /home/debian-wazu#

```



## Résultat :

- ✓ **Source et/open activée (Emerging Threats Open)**
- ✓ **Fournisseur** → Proofpoint
- ✓ **Licence** → MIT (Open Source)
- ✓ **Coût** → Gratuit
- ✓ **Répertoire créé** → `/var/lib/suricata/update/sources`
- ✓ **Mise à jour** → Quotidienne (nouvelles menaces)
- ✓ **Couverture** → Large spectre de menaces connues

La source *Emerging Threats Open*, maintenue par *Proofpoint*, fournit un ensemble complet de règles de détection gratuites et régulièrement mises à jour, couvrant les vulnérabilités récentes, les malwares connus et les comportements suspects sur le réseau.

## 4.2 Sources de règles disponibles

L'écosystème Suricata propose plusieurs sources de règles, chacune adaptée à des besoins et des budgets différents.

Screenshot 14 - Liste des sources disponibles (et/open, et/pro, oisf/trafficid) avec `suricata-update list-sources`

```

root@debian-wazu: /home/debian-wazu# suricata-update list-sources
18/11/2025 -- 21:56:38 - <Info> -- Using data-directory /var/lib/suricata.
18/11/2025 -- 21:56:38 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
18/11/2025 -- 21:56:38 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
18/11/2025 -- 21:56:38 - <Info> -- Found Suricata version 6.0.10 at /usr/bin/suricata.
18/11/2025 -- 21:56:38 - <Info> -- No source index found, running update-sources
18/11/2025 -- 21:56:38 - <Info> -- Downloading https://www.openinfosecfoundation.org/rules/index.yaml
18/11/2025 -- 21:56:39 - <Info> -- Adding all sources
18/11/2025 -- 21:56:39 - <Info> -- Saved /var/lib/suricata/update/cache/index.yaml

Name: et/open
  Vendor: Proofpoint
  Summary: Emerging Threats Open Ruleset
  License: MIT

Name: et/pro
  Vendor: Proofpoint
  Summary: Emerging Threats Pro Ruleset
  License: Commercial
  Replaces: et/open
  Parameters: secret-code
  Subscription: https://www.proofpoint.com/us/threat-insight/et-pro-ruleset

Name: oisf/trafficid
  Vendor: OISF
  Summary: Suricata Traffic ID ruleset
  License: MIT

```





**Sources de règles identifiées :****1. et/open (Proofpoint) ✓ Activé**

- **Description** → *Emerging Threats Open Ruleset*
- **Licence** → MIT (Open source)
- **Coût** → Gratuit
- **État** → Activé
- **Utilisation** → Production recommandée

**2. et/pro (Proofpoint)**

- **Description** → *Emerging Threats Pro Ruleset*
- **Licence** → Commercial
- **Coût** → Abonnement payant
- **Avantages** → Règles avancées, mises à jour plus fréquentes
- **Paramètres requis** → secret-code (clé d'abonnement)
- **Site** → <https://www.proofpoint.com/us/threat-insight/et-pro-ruleset>

**3. oisf/trafficid (OISF)**

- **Description** → *Suricata Traffic ID ruleset*
- **Licence** → MIT (Open Source)
- **Coût** → Gratuit
- **Spécialité** → Identification et classification du trafic

Pour ce projet, la source *et/open* est suffisante et constitue une base solide pour la détection.

**5. Déploiement et activation des règles****5.1 Processus de mise à jour initiale**

La première exécution de *suricata-update* télécharge et configure l'ensemble complet des règles de détection depuis la source activée.



Screenshot 15 - Téléchargement des règles (5,2 Mo depuis [emergingthreats.net](https://rules.emergingthreats.net/open/suricata-6.0.10/emerging.rules.tar.gz)) avec `suricata-update`

```

root@debian-wazu: /home/debian-wazu# suricata-update
18/11/2025 -- 21:55:54 - <Info> -- Using data-directory /var/lib/suricata.
18/11/2025 -- 21:55:54 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
18/11/2025 -- 21:55:54 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
18/11/2025 -- 21:55:54 - <Info> -- Found Suricata version 6.0.10 at /usr/bin/suricata.
18/11/2025 -- 21:55:54 - <Info> -- Loading /etc/suricata/suricata.yaml
18/11/2025 -- 21:55:54 - <Info> -- Disabling rules for protocol http2
18/11/2025 -- 21:55:54 - <Info> -- Disabling rules for protocol modbus
18/11/2025 -- 21:55:54 - <Info> -- Disabling rules for protocol dnp3
18/11/2025 -- 21:55:54 - <Info> -- Disabling rules for protocol enip
18/11/2025 -- 21:55:54 - <Info> -- No sources configured, will use Emerging Threats Open
18/11/2025 -- 21:55:54 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-6.0.10/em
erging.rules.tar.gz.
100% - 5191173/5191173
18/11/2025 -- 21:55:58 - <Info> -- Done.
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/app-layer-event
s.rules
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/decoder-events.
rules
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/dhcp-events.rul
es
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/dnp3-events.rul
es
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/dns-events.rule

```

## Processus détaillé :

### 1. Chargement de la configuration

- Source → `/etc/suricata/suricata.yaml`

### 2. Identification de la version

- Suricata → `6.0.10`
- Répertoire de règles → `/etc/suricata/rules`

### 3. Optimisation automatique Désactivation des protocoles non surveillés :

- HTTP/2 (http2)
- Modbus
- DNP3
- ENIP

### 4. Téléchargement

- Source → *Emerging Threats Open*



- URL → <https://rules.emergingthreats.net/open/suricata-6.0.10/emerging.rules.tar.gz>
- Taille → 5,2 Mo
- Format → *tar.gz*

Cette optimisation automatique réduit la charge de traitement en évitant l'analyse de protocoles non pertinents pour l'environnement surveillé.

## 5.2 Statistiques de déploiement des règles

Screenshot 16 - Déploiement de 46334 règles actives sur 62149 chargées avec *suricata-update* (suite)

```

s
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/ntp-events.rule
s
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/smb-events.rule
s
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/smtp-events.rul
es
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/stream-events.r
ules
18/11/2025 -- 21:55:59 - <Info> -- Loading distribution rule file /etc/suricata/rules/tls-events.rule
s
18/11/2025 -- 21:55:59 - <Info> -- Ignoring file rules/emerging-deleted.rules
18/11/2025 -- 21:56:01 - <Info> -- Loaded 62149 rules.
18/11/2025 -- 21:56:01 - <Info> -- Disabled 14 rules.
18/11/2025 -- 21:56:01 - <Info> -- Enabled 0 rules.
18/11/2025 -- 21:56:01 - <Info> -- Modified 0 rules.
18/11/2025 -- 21:56:01 - <Info> -- Dropped 0 rules.
18/11/2025 -- 21:56:01 - <Info> -- Enabled 136 rules for flowbit dependencies.
18/11/2025 -- 21:56:01 - <Info> -- Backing up current rules.
18/11/2025 -- 21:56:01 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 62
149; enabled: 46334; added: 62149; removed 0; modified: 0
18/11/2025 -- 21:56:02 - <Info> -- Writing /var/lib/suricata/rules/classification.config
18/11/2025 -- 21:56:02 - <Info> -- Testing with suricata -T.
18/11/2025 -- 21:56:24 - <Info> -- Done.

```

### Statistiques de déploiement :

- **Règles chargées** → 62 149
- **Règles désactivées (protocoles non utilisés)** → 14
- **Règles explicitement activées (première installation)** → 0
- **Règles modifiées** → 0
- **Règles supprimées** → 0



➤ **Règles flowbit auto-activées (dépendances de corrélation d'événements)** →

136

**Total règles actives → 46 334 (74,6%)****Fichiers générés :**

- `/var/lib/suricata/rules/suricata.rules` → Contient les 46 334 règles actives
- `/var/lib/suricata/rules/classification.config` → Définit les catégories et priorités des alertes

**Note sur les flowbits** → Les *flowbits* sont des mécanismes permettant de corréler plusieurs événements réseau. Certaines règles nécessitent que d'autres règles soient activées pour fonctionner correctement. L'activation automatique de 136 règles garantit que toutes les détections basées sur des corrélations fonctionneront comme prévu.

### 5.3 Validation de la configuration

Avant de démarrer Suricata en production, un test complet de configuration est indispensable pour identifier et corriger toute erreur potentielle.

*Screenshot 17 - Validation de la configuration avec suricata -T -c /etc/suricata/suricata.yaml -v*

```
root@debian-wazu: /home/debian-wazu# micro /etc/suricata/suricata.yaml
root@debian-wazu: /home/debian-wazu# suricata -T -c /etc/suricata/suricata.yaml -v
18/11/2025 -- 22:00:41 - <Info> - Running suricata under test mode
18/11/2025 -- 22:00:41 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
18/11/2025 -- 22:00:41 - <Info> - CPUs/cores online: 2
18/11/2025 -- 22:00:41 - <Info> - fast output device (regular) initialized: fast.log
18/11/2025 -- 22:00:41 - <Info> - eve-log output device (regular) initialized: eve.json
18/11/2025 -- 22:00:41 - <Info> - stats output device (regular) initialized: stats.log
18/11/2025 -- 22:00:41 - <Warning> - [ERRCODE: SC_ERR_NO_RULES(42)] - No rule files match the pattern /etc/suricata/rules/suricata.rules
root@debian-wazu: /home/debian-wazu#
```

Le test de configuration a validé que tous les fichiers sont syntaxiquement corrects, que les règles se chargent sans erreur et que les paramètres réseau sont cohérents. Le test a confirmé le chargement de 46 334 règles avec succès, l'initialisation des fichiers de sortie (*fast.log*, *eve.json*, *stats.log*) et l'absence d'erreurs. Cette validation est cruciale avant le lancement en production.

**Résultat** → Configuration valide (*Done*)

## 6. Synthèse de l'installation

### 6.1 Composants logiciels déployés

Le système Debian 12 dispose désormais d'une installation complète et fonctionnelle de Suricata avec tous les outils nécessaires à son fonctionnement optimal et son intégration dans l'écosystème de sécurité.



## Versions installées :

- **Suricata** → 6.0.10
- **jq** → 1.6-2.1
- **software-properties-common** → 0.99.30-4.1
- **python3-pip** → Dernière version disponible

## 6.2 Architecture des fichiers de configuration

### Fichiers importants :

L'organisation des fichiers suit les standards Debian, facilitant la maintenance et la compréhension du système.

| Fichier                  | Emplacement                                   |
|--------------------------|-----------------------------------------------|
| Configuration principale | /etc/suricata/suricata.yaml                   |
| Règles Suricata          | /var/lib/suricata/rules/suricata.rules        |
| Règles distribuées       | /etc/suricata/rules/                          |
| Classification           | /var/lib/suricata/rules/classification.config |
| Logs JSON                | /var/log/suricata/eve.json                    |
| Cache des sources        | /var/lib/suricata/update/cache/               |

## 6.3 État opérationnel du système

Le système est stable et validé, prêt pour le mode opérationnel. La commande *suricata -T* a confirmé l'absence d'erreurs. Les 46 334 règles actives couvrent un large spectre de menaces (exploitation de vulnérabilités, serveurs de commande et contrôle). Le moteur de capture surveille l'interface *ens192* avec *multi-threading* optimisé. La génération de logs JSON est prête pour l'intégration avec Wazuh.





## 7. Intégration de Suricata avec Wazuh

### 7.1 Configuration du fichier *ossec.conf*

L'intégration nécessite la configuration de */var/ossec/etc/ossec.conf* pour surveiller les logs *JSON* de Suricata. Le fichier a été édité avec *micro* en ajoutant un bloc *localfile*. Le format de log « *json* » indique à Wazuh un traitement *JSON* spécifique. L'emplacement */var/log/suricata/eve.json* correspond au fichier où Suricata enregistre ses événements EVE. Cette configuration permet à l'agent Wazuh de lire en continu et transmettre les événements au serveur pour analyse.

*Screenshot 18 - Édition de la configuration Wazuh avec micro /var/ossec/etc/ossec.conf*

```
root@debian-wazu:/home/debian-wazu# micro /var/ossec/etc/ossec.conf
```

**Configuration ajoutée dans */var/ossec/etc/ossec.conf* :**

*Screenshot 19 - Configuration localfile (eve.json, format JSON) - ossec.conf lignes 346-351*

```
346 <localfile>
347   <log_format>json</log_format>
348   <location>/var/log/suricata/eve.json</location>
349 </localfile>
350
351 </ossec_config>
```

### 7.2 Résolution des problèmes de décodage *JSON*

Après la première tentative d'intégration, de nombreuses erreurs « *wazuh-analysisd: ERROR: Too many fields for JSON decoder* » sont apparues, se répétant continuellement. Ces erreurs indiquaient que les événements *JSON* de Suricata contenaient plus de champs que la limite par défaut du décodeur *JSON* de Wazuh.

*Screenshot 20 - Événements "stats" volumineux causant les erreurs avec tail -f /var/ossec/logs/ossec.log*

```
root@debian-wazu:/home/debian-wazu# tail -f /var/ossec/logs/ossec.log
2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.
2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.
2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.
2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.
2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.
```

**Erreurs observées :**

*2025/11/18 22:15:10 wazuh-analysisd: ERROR: Too many fields for JSON decoder.*

**Résolution définitive du problème :**

Pour éliminer les erreurs de décodage *JSON*, la limite de champs du décodeur Wazuh a été augmentée.



## Édition du fichier de configuration interne :

Screenshot 21 - Édition du fichier *internal\_options.conf* avec *micro* */var/ossec/etc/internal\_options.conf*

```
root@debian-wazu:/home/debian-wazu# micro /var/ossec/etc/internal_options.conf
```

## Modification effectuée (ligne 30) :

La valeur par défaut du paramètre *analysisd.decoder\_order\_size* a été augmentée à 512 pour permettre le traitement des événements *JSON* volumineux de Suricata.

Screenshot 22 - Configuration *analysisd.decoder\_order\_size=512* - *internal\_options.conf* ligne 30

```
1 # internal_options.conf, Daniel B. Cid (dcid @ ossec.net).
2 #
3 # DO NOT TOUCH THIS FILE. The default configuration
4 # is at ossec.conf. More information at:
5 # https://documentation.wazuh.com
6 #
7 # This file should be handled with care. It contain
8 # run time modifications that can affect the use
9 # of ossec. Only change it if you know what you
10 # are doing. Again, look first at ossec.conf
11 # for most of the things you want to change.
12
13
14 # Analysisd default rule timeframe.
15 analysisd.default_timeframe=360
16 # Analysisd stats maximum diff.
17 analysisd.stats_maxdiff=999000
18 # Analysisd stats minimum diff.
19 analysisd.stats_mindiff=1250
20 # Analysisd stats percentage (how much to differ from average)
21 analysisd.stats_percent_diff=150
22 # Analysisd FTS list size.
23 analysisd.fts_list_size=32
24 # Analysisd FTS minimum string size.
25 analysisd.fts_min_size_for_str=14
26 # Analysisd Enable the firewall log (at logs/firewall/firewall.log)
27 # 1 to enable, 0 to disable.
28 analysisd.log_fw=1
29 # Maximum number of fields in a decoder (order tag) [32..1024]
30 analysisd.decoder_order_size=512
31 # Output GeoIP data at JSON alerts
32 analysisd.geoip_jsonout=0
33 # Maximum label cache age (margin seconds with no reloading) [0..60]
34 analysisd.label_cache_maxage=10
35 # Show hidden labels on alerts
36 analysisd.show_hidden_labels=0
37 # Maximum number of file descriptor that Analysisd can open [1024..1048576]
```

## Paramètres visibles dans le fichier :

| Paramètre                           | Valeur | Description                      |
|-------------------------------------|--------|----------------------------------|
| <i>analysisd.default_timeframe</i>  | 360    | Fenêtre temporelle par défaut    |
| <i>analysisd.stats_maxdiff</i>      | 999000 | Différence maximale statistiques |
| <i>analysisd.decoder_order_size</i> | 512    | Nombre maximum de champs JSON    |
| <i>analysisd.log_fw</i>             | 1      | Journalisation firewall activée  |



Cette modification permet au décodeur *JSON* de Wazuh de traiter l'intégralité des événements Suricata, y compris les événements « *stats* » volumineux, sans générer d'erreurs.

Screenshot 23 - Exemple d'événement DNS formaté JSON avec `tail -50 /var/log/suricata/eve.json | jq`

```

{
  "timestamp": "2025-11-18T22:20:05.130231+0100",
  "flow_id": 1277812413954975,
  "in_iface": "ens192",
  "event_type": "dns",
  "src_ip": "192.168.111.62",
  "src_port": 36919,
  "dest_ip": "192.168.111.250",
  "dest_port": 53,
  "proto": "UDP",
  "dns": {
    "version": 2,
    "type": "answer",
    "id": 58277,
    "flags": "8180",
    "qr": true,
    "rd": true,
    "ra": true,
    "rrname": "mail.google.com",
    "rrtype": "AAAA",
    "rcode": "NOERROR",
    "answers": [
      {
        "rrname": "mail.google.com",
        "rrtype": "AAAA",
        "ttl": 128,
        "rdata": "2404:6800:4006:0814:0000:0000:0000:2005"
      }
    ]
  },
  "grouped": {
    "AAAA": [
      "2404:6800:4006:0814:0000:0000:0000:2005"
    ]
  }
}
{
  "timestamp": "2025-11-18T22:20:05.264468+0100",
  "event_type": "stats",
  "stats": {
    "packets": 1,
    "bytes": 128,
    "errors": 0,
    "protocol": "UDP",
    "source_ip": "192.168.111.62",
    "destination_ip": "192.168.111.250",
    "source_port": 36919,
    "destination_port": 53
  }
}

```

L'examen du fichier *eve.json* a confirmé la richesse des informations. Un événement DNS typique contient → timestamp, interface réseau, type d'événement, adresses IP source/destination, ports, protocole, détails DNS (type de requête, réponses, codes de retour).

Les événements « *stats* » sont encore plus volumineux, contenant des statistiques détaillées → compteurs de paquets traités, erreurs réseau, statistiques par protocole, métriques de mémoire et nombreuses informations techniques. Ces événements généraient systématiquement les erreurs de décodage.

## 7.3 Correction des permissions et redémarrage

Les permissions du fichier *eve.json* ont été corrigées pour que Wazuh puisse le lire correctement. Les commandes `chmod 644` définissent les permissions en lecture pour tous et `chown root:root` attribue la propriété au compte root. Le service Wazuh Manager a été redémarré pour appliquer les modifications et réinitialiser le décodeur JSON.

Screenshot 24 - Correction des permissions avec `chmod 644 && chown root:root && systemctl restart`

```

root@debian-wazu:/home/debian-wazu# chmod 644 /var/log/suricata/eve.json
root@debian-wazu:/home/debian-wazu# chown root:root /var/log/suricata/eve.json
root@debian-wazu:/home/debian-wazu# systemctl restart wazuh-manager

```



## 7.4 Vérification de l'intégration

La vérification a confirmé que *wazuh-logcollector* analyse bien le fichier */var/log/suricata/eve.json*. Les entrées de log datées du 18/11/2025 à 22:14:30 montrent que le composant logcollector (ID 1950) a pris en charge l'analyse du fichier.

Malgré quelques erreurs de décodage persistantes (événements « stats »), l'intégration de base était fonctionnelle pour les alertes de sécurité.

### Commandes de vérification :

```
grep -i suricata /var/ossec/logs/ossec.log
```

```
grep -i "eve.json" /var/ossec/logs/ossec.log
```

Screenshot 25 - Vérification de l'intégration

```
root@debian-wazu:/home/debian-wazu# grep -i suricata /var/ossec/logs/ossec.log
2025/11/18 22:14:30 wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/suricata/eve.json'.
root@debian-wazu:/home/debian-wazu# grep -i "eve.json" /var/ossec/logs/ossec.log
2025/11/18 22:14:30 wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/suricata/eve.json'.
root@debian-wazu:/home/debian-wazu# █
```

## 8. Tests de détection et validation

### 8.1 Génération d'événements de test

Des tests ont été effectués avec *curl* vers *testmyids.org* pour déclencher des alertes IDS. Le test a été effectué en deux temps avec une pause de 2 secondes. La première requête vers *testmyids.org/uid/index.html* visait à déclencher une règle de détection spécifique. La deuxième requête vers *testmyids.com* a renvoyé *uid=0(root) gid=0(root) groups=0(root)*, confirmant l'exécution avec privilèges root.

### Commandes exécutées :

Screenshot 26 - Génération d'événements de test

```
root@debian-wazu:/home/debian-wazu# curl http://testmyids.org/uid/index.html
root@debian-wazu:/home/debian-wazu# sleep 2
root@debian-wazu:/home/debian-wazu# curl http://testmyids.com/
uid=0(root) gid=0(root) groups=0(root)
root@debian-wazu:/home/debian-wazu#
```

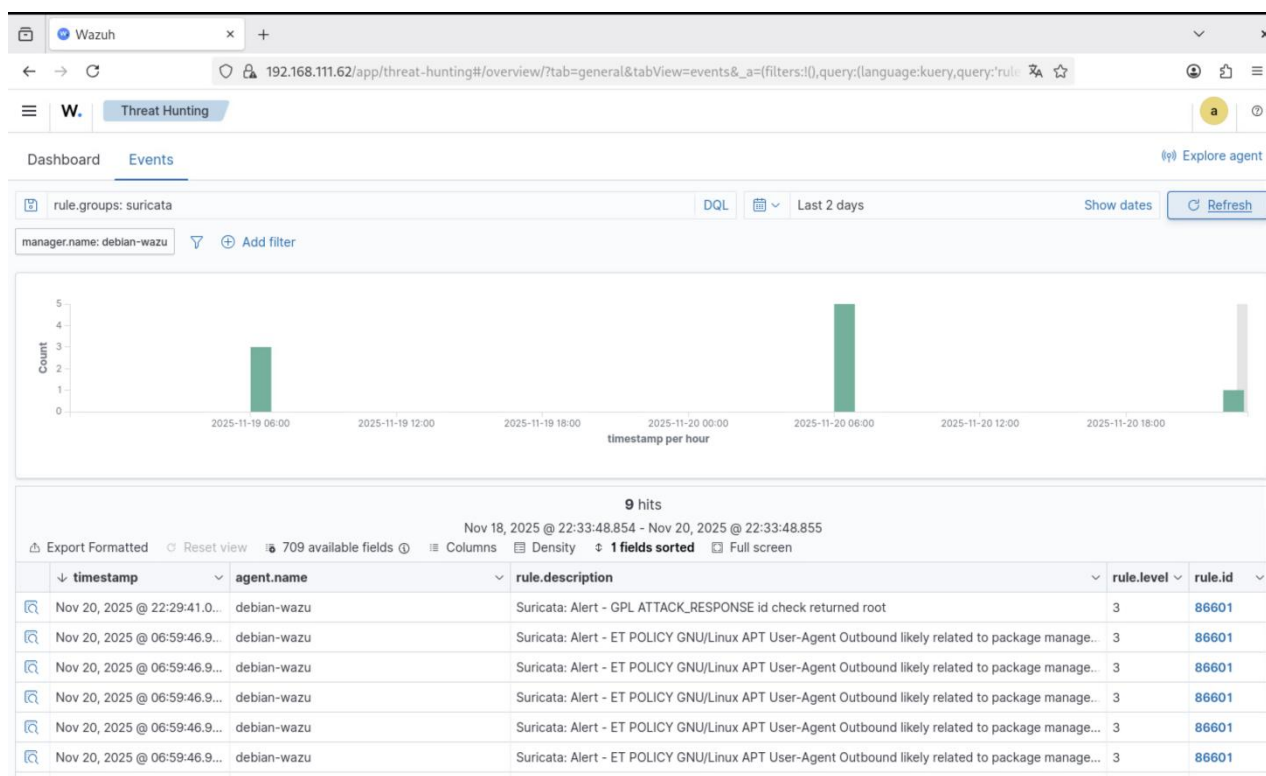
**Résultat** → uid=0(root) gid=0(root) groups=0(root)

### 8.2 Détection des alertes dans Wazuh

L'interface web Wazuh (*192.168.111.62/app/threat-hunting*) affiche les événements en temps réel avec options de filtrage avancées. Un filtre a été appliqué pour afficher uniquement les événements de l'agent « *debian-wazu* » et du groupe de règles « *suricata* ». Le graphique temporel montre une alerte détectée vers 22h29.



Screenshot 27 - Visualisation des alertes dans l'interface Wazuh Threat Hunting



## Résultat affiché :

- 9 hits détectés
- Période → 18/11/2025 22:33:48 - 20/11/2025 22:33:48
- Types d'alertes détectées :
  - GPL ATTACK\_RESPONSE id check returned root
  - ET POLICY GNU/Linux APT User-Agent Outbound likely related to package manager
- Niveau → 3
- Rule ID → 86601

Le graphique temporel montre l'activité de détection sur 2 jours, avec des pics d'alertes le 19/11 vers 06:00 et le 20/11 vers 06:00, correspondant aux mises à jour système automatiques (alertes APT User-Agent).





### 8.3 Analyse détaillée de l'alerte

L'alerte détaillée affiche toutes les informations structurées → index wazuh-alerts-4.x-2025.11.21, agent ID 000 (debian-wazu), action « *allowed* » (détection seule, pas de blocage), catégorie « *Potentially Bad Traffic* », confiance « *Medium* », sévérité de signature « *Informational* ». La signature *Suricata ID 2100498* correspond à la règle « *GPL ATTACK\_RESPONSE id check returned root* », créée le 23/09/2010, mise à jour le 26/07/2019, révision 7. Le protocole application est HTTP avec communication depuis 192.168.111.62 vers le port de destination 35436.

Screenshot 28 - Métadonnées complètes de l'alerte GPL ATTACK\_RESPONSE dans Document Details Wazuh

Document Details
View surrounding documents
View single document

Table
JSON

|                                          |                                            |
|------------------------------------------|--------------------------------------------|
| † _index                                 | wazuh-alerts-4.x-2025.11.21                |
| † agent.id                               | 000                                        |
| † agent.name                             | debian-wazu                                |
| † data.alert.action                      | allowed                                    |
| † data.alert.category                    | Potentially Bad Traffic                    |
| † data.alert.gid                         | 1                                          |
| † data.alert.metadata.confidence         | Medium                                     |
| † data.alert.metadata.created_at         | 2010_09_23                                 |
| † data.alert.metadata.signature_severity | Informational                              |
| † data.alert.metadata.updated_at         | 2019_07_26                                 |
| † data.alert.rev                         | 7                                          |
| † data.alert.severity                    | 2                                          |
| † data.alert.signature                   | GPL ATTACK_RESPONSE id check returned root |
| † data.alert.signature_id                | 2100498                                    |
| † data.app_proto                         | http                                       |
| † data.dest_ip                           | 192.168.111.62                             |
| † data.dest_port                         | 33880                                      |



## 9. Configuration et démarrage du service Suricata

### 9.1 Activation du démarrage automatique

Le service *systemd* a été configuré pour démarrage automatique. La commande *systemctl enable* a créé les liens symboliques nécessaires dans l'arborescence *systemd* en synchronisant avec le script SysV.

*Screenshot 29 - Activation du service avec systemctl enable suricata*

```
root@debian-wazu:/home/debian-wazu# systemctl enable suricata
Synchronizing state of suricata.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable suricata
```

**Résultat** → Service activé pour démarrage automatique

### 9.2 Démarrage et vérification du service

Le service a été démarré manuellement et son état vérifié. Le statut affiche que le service est actif et en cours d'exécution depuis le 18/11/2025 à 22:08:40 CET. Le processus principal porte le PID 1200378, exécutant la commande complète avec *mode daemon (-D)*, *capture AF\_PACKET (--af-packet)*, *configuration /etc/suricata/suricata.yaml (-c)* et *pidfile /run/suricata.pid*. Le processus Suricata-Main (PID 1200379) est limité à 9447 tâches, utilise 304,2 Mo de mémoire et consomme 6,732 secondes CPU.

*Screenshot 30 - Démarrage et vérification du service avec systemctl start suricata ; systemctl status Suricata*

```
root@debian-wazu:/home/debian-wazu# systemctl start suricata
root@debian-wazu:/home/debian-wazu# systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-11-18 22:08:40 CET; 7s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata-ids.org/docs/
   Process: 1200378 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid (code=exited, status=0/SUCCESS)
  Main PID: 1200379 (Suricata-Main)
    Tasks: 1 (limit: 9447)
   Memory: 304.2M
      CPU: 6.732s
   CGroup: /system.slice/suricata.service
           └─1200379 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

nov. 18 22:08:40 debian-wazu systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
nov. 18 22:08:40 debian-wazu suricata[1200378]: 18/11/2025 -- 22:08:40 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
nov. 18 22:08:40 debian-wazu systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
root@debian-wazu:/home/debian-wazu#
```

#### État du service :

- **Status** → active (running)
- **Démarré** → 18/11/2025 22:08:40 CET
- **PID principal** → 1200379 (Suricata-Main)
- **Mémoire** → 304,2 Mo
- **CPU** → 6,732s



➤ **Tâches** → 1 (limite : 9447)

## 9.3 Automatisation des mises à jour des règles

Pour maintenir les signatures de détection à jour automatiquement, une tâche cron a été configurée.

*Screenshot 31 - Création du crontab avec sélection de l'éditeur nano*

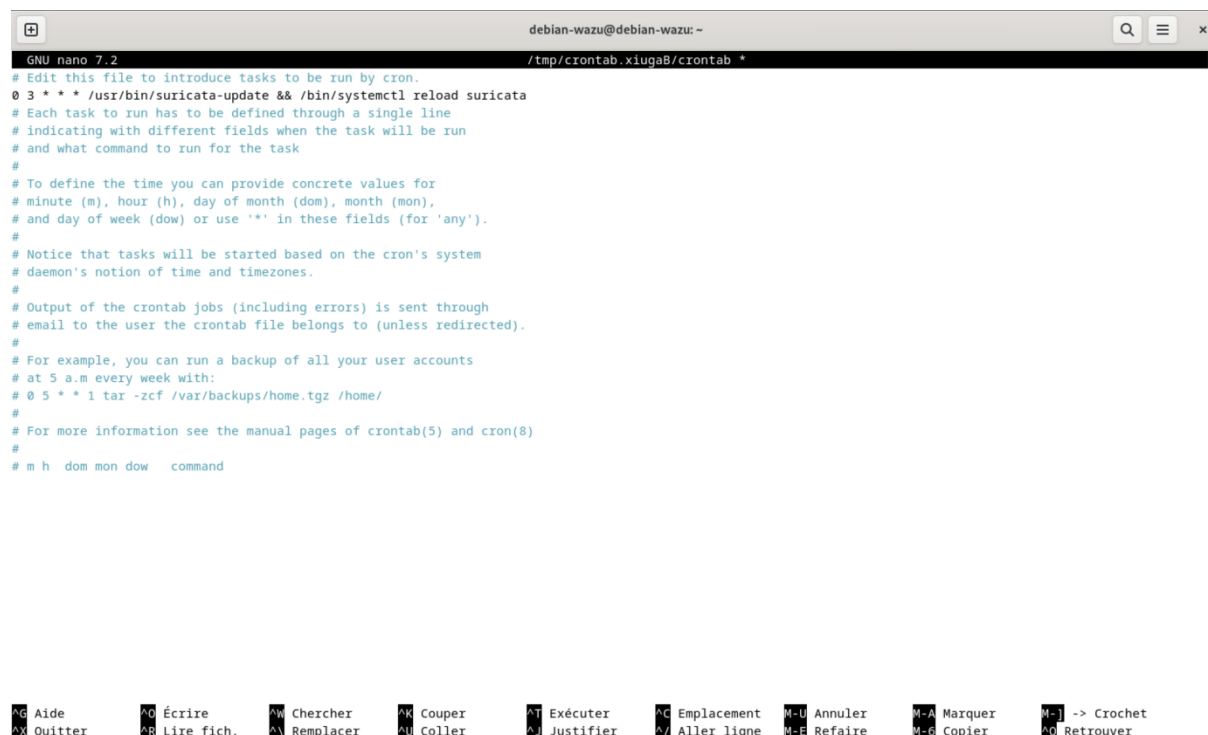
```
root@debian-wazu:/home/debian-wazu# crontab -e
no crontab for root - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /usr/bin/micro
 2. /bin/nano      <---- easiest
 3. /usr/bin/vim.tiny

Choose 1-3 [2]: 2
crontab: installing new crontab
```

### Configuration ajoutée :

*Screenshot 32 - Édition du crontab dans nano avec la tâche suricata-update*



```
GNU nano 7.2 /tmp/crontab.xiuga8/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 3 * * * /usr/bin/suricata-update && /bin/systemctl reload suricata
```

### Tâche planifiée :

```
0 3 * * * /usr/bin/suricata-update && /bin/systemctl reload suricata
```



### Explication de la syntaxe :

| <i>Champ</i>              | <i>Valeur</i> | <i>Signification</i> |
|---------------------------|---------------|----------------------|
| <i>Minute</i>             | 0             | À la minute 0        |
| <i>Heure</i>              | 3             | À 3h du matin        |
| <i>Jour du mois</i>       | *             | Tous les jours       |
| <i>Mois</i>               | *             | Tous les mois        |
| <i>Jour de la semaine</i> | *             | Tous les jours       |

### Vérification de la configuration :

Screenshot 33 - Vérification du crontab avec crontab -l

```

root@debian-wazu:/home/debian-wazu# crontab -l
# Edit this file to introduce tasks to be run by cron.
# 0 3 * * * /usr/bin/suricata-update && /bin/systemctl reload suricata
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
root@debian-wazu:/home/debian-wazu# █

```

Cette configuration assure une mise à jour quotidienne des règles *Emerging Threats* à 3h00 du matin, suivie d'un rechargement à chaud de Suricata sans interruption du service de surveillance.

### 9.4 Configuration de la rotation des logs

Le système dispose d'une configuration de rotation des logs préinstallée pour éviter la saturation du disque.



Screenshot 34 - Édition du fichier de configuration des rotations avec micro /etc/logrotate.d/suricata

```
root@debian-wazu:/home/debian-wazu# micro /etc/logrotate.d/suricata
```

## Fichier de configuration :

Screenshot 35 - Configuration logrotate pour Suricata

```

1 /var/log/suricata/*.log
2 /var/log/suricata/*.json
3 {
4     rotate 14
5     missingok
6     compress
7     copytruncate
8     sharedscripts
9     postrotate
10         /bin/kill -HUP $(cat /var/run/suricata.pid)
11     endscript
12 }
13

```

## Paramètres configurés :

| Directive            | Fonction                                 |
|----------------------|------------------------------------------|
| <b>rotate 14</b>     | Conservation de 14 jours d'historique    |
| <b>missingok</b>     | Pas d'erreur si fichier absent           |
| <b>compress</b>      | Compression des anciens fichiers         |
| <b>copytruncate</b>  | Copie puis vide le fichier actif         |
| <b>sharedscripts</b> | Exécution unique du script post-rotation |
| <b>postrotate</b>    | Signal HUP envoyé à Suricata             |

Cette configuration assure une gestion automatique de l'espace disque avec conservation de deux semaines d'historique compressé.

## 10. Vérification et analyse des règles

### 10.1 Inspection du répertoire des règles

Le répertoire `/var/lib/suricata/rules/` contient les règles compilées et prêtes. Le fichier `suricata.rules` pèse 40 519 950 octets (38,6 Mo) et le fichier `classification.config` occupe 3228 octets. Les horodatages du 18/11 à 21:58 correspondent à la dernière exécution de `suricata-update`, confirmant la cohérence de la mise à jour.



## Screenshot 36 - Vérification des fichiers avec `ls -la /var/lib/suricata/rules/`

```
root@debian-wazu:/home/debian-wazu# ls -la /var/lib/suricata/rules/
total 39584
drwxr-xr-x 2 root root 4096 18 nov. 21:58 .
drwxr-xr-x 4 root root 4096 18 nov. 21:55 ..
-rw-r--r-- 1 root root 3228 18 nov. 21:58 classification.config
-rw-r--r-- 1 root root 40519950 18 nov. 21:58 suricata.rules
root@debian-wazu:/home/debian-wazu# micro /etc/suricata/suricata.yaml
root@debian-wazu:/home/debian-wazu# suricata -T -c /etc/suricata/suricata.yaml -v
18/11/2025 -- 22:07:09 - <Info> - Running suricata under test mode
18/11/2025 -- 22:07:09 - <Notice> - This is Suricata version 6.0.10 RELEASE running in SYSTEM mode
18/11/2025 -- 22:07:09 - <Info> - CPUs/cores online: 2
18/11/2025 -- 22:07:09 - <Info> - fast output device (regular) initialized: fast.log
18/11/2025 -- 22:07:09 - <Info> - eve-log output device (regular) initialized: eve.json
18/11/2025 -- 22:07:09 - <Info> - stats output device (regular) initialized: stats.log
18/11/2025 -- 22:07:18 - <Info> - 1 rule files processed. 46334 rules successfully loaded, 0 rules failed
18/11/2025 -- 22:07:18 - <Info> - Threshold config parsed: 0 rule(s) found
18/11/2025 -- 22:07:19 - <Info> - 46337 signatures processed. 972 are IP-only rules, 5158 are inspecting packet payload, 39990 inspect application layer, 108
are decoder event only
18/11/2025 -- 22:07:31 - <Notice> - Configuration provided was successfully loaded. Exiting.
18/11/2025 -- 22:07:31 - <Info> - cleaning up signature grouping structure... complete
root@debian-wazu:/home/debian-wazu#
```

## Contenu du répertoire :

- **suricata.rules** → 38,6 Mo
- **classification.config** → 3,2 Ko
- **Dernière mise à jour** → 18/11 21:58

## Conclusion

### Synthèse technique

L'installation et la configuration de Suricata 6.0.10 sur Debian 12 ont été réalisées avec succès. Le système surveille désormais le trafic réseau de l'interface `ens192` (192.168.111.0/24) avec 46 334 règles actives provenant de la source *Emerging Threats Open*.

### Points clés de l'implémentation

- **Composants installés :**
  - Suricata 6.0.10 avec support *AF\_PACKET*
  - Configuration optimisée pour 2 threads de traitement
  - 46 334 règles de détection actives (74,6% du total disponible)
  - Intégration complète avec Wazuh via logs *EVE-JSON*
- **Architecture réseau :**
  - **Interface surveillée** → `ens192` (192.168.111.0/24)
  - **Mode de capture** → *AF\_PACKET* avec `cluster_flow`





- **Zones définies** → *HOME\_NET* (réseau interne) et *EXTERNAL\_NET* (trafic externe)
- **Règles de détection** →
  - **Source** → *Emerging Threats Open (Proofpoint)*
  - **Mise à jour** → Quotidienne automatique
  - **Couverture** → Vulnérabilités, malwares, comportements suspects

## Intégration Wazuh

**L'intégration avec Wazuh est opérationnelle et fonctionnelle :**

- ☒ Configuration *ossec.conf* validée
- ☒ Logs *EVE-JSON* correctement analysés
- ☒ Alertes remontées en temps réel dans l'interface Wazuh
- ☒ Tests de détection réussis avec *testmyids.org*

## Tests et validation

**Les tests effectués ont confirmé le bon fonctionnement du système :**

- ☒ Configuration validée avec *suricata -T*
- ☒ Service *systemd* actif et fonctionnel
- ☒ Détection d'attaques simulées réussie
- ☒ Alerte *GPL ATTACK\_RESPONSE* correctement générée et remontée dans Wazuh
- ☒ Niveau de confiance et catégorisation appropriés

## Performances système

**Le système fonctionne avec des performances optimales :**

- **Consommation mémoire** → 304,2 Mo
- **Utilisation CPU** → 6,732s cumulées
- **Mode multi-threading** → 2 threads actifs
- **Latence de capture** → Minimisée avec *use-mmap*



## **Recommandations et prochaines étapes**

### *Personnalisation des règles*

#### **Adapter les règles de détection aux besoins spécifiques de l'environnement :**

- Créer des règles personnalisées dans `/etc/suricata/rules/local.rules`
- Désactiver les règles générant trop de faux positifs
- Ajuster les seuils de détection selon le contexte métier

### *Configuration des alertes*

#### **Mettre en place un système de notification pour les événements critiques :**

- Configurer les alertes email dans Wazuh
- Définir les niveaux de priorité des alertes
- Créer des tableaux de bord personnalisés dans Wazuh

### *Documentation des procédures*

#### **Créer des runbooks pour les opérations courantes :**

- Procédure de mise à jour de Suricata
- Résolution des problèmes courants
- Ajout de nouvelles règles personnalisées
- Analyse des faux positifs

### *Tests de charge*

#### **Évaluer les performances sous trafic élevé :**

- Simuler une charge réseau importante
- Vérifier la stabilité du système
- Ajuster les paramètres si nécessaire (threads, mémoire)

### *Supervision continue*

#### **Mettre en place une surveillance proactive du système :**

- Monitorer l'utilisation CPU et mémoire de Suricata



- Surveiller la taille des fichiers de logs
- Vérifier régulièrement l'état du service
- Analyser les performances de détection

### Formation et sensibilisation

#### Former l'équipe de sécurité :

- Interprétation des alertes Suricata
- Utilisation de l'interface *Wazuh Threat Hunting*
- Réponse aux incidents détectés
- Analyse des logs *EVE-JSON*

### État final du système

Le système *Suricata-Wazuh* est **opérationnel et prêt pour la production**.

#### Capacités de détection actives :

- ☒ 46 334 règles de détection d'intrusions
- ☒ Surveillance réseau temps réel sur *192.168.111.0/24*
- ☒ Détection de vulnérabilités et exploits
- ☒ Identification de malwares et trojans
- ☒ Détection de communications C&C (Command & Control)
- ☒ Analyse de comportements suspects
- ☒ Journalisation complète en format *JSON*

#### Couverture de sécurité :

- **Périmètre surveillé** → Réseau interne *192.168.111.0/24*
- **Interface** → *ens192*
- **Protocoles analysés** → HTTP, DNS, TLS, SSH, FTP, SMTP, et protocoles industriels



- **Mode de fonctionnement** → Détection (IDS) avec possibilité d'évolution vers prévention (IPS)

## Intégration SIEM :

- **Plateforme** → Wazuh 4.x
- **Format de logs** → *EVE-JSON* structuré
- **Corrélation** → Active avec les autres sources Wazuh
- **Interface** → *Threat Hunting* accessible via web

## Conformité et bonnes pratiques

### Cette installation respecte les bonnes pratiques de sécurité :

- ☒ Système à jour avec les derniers correctifs
- ☒ Règles de détection maintenues par *Proofpoint*
- ☒ Séparation des zones réseau (*HOME\_NET* / *EXTERNAL\_NET*)
- ☒ Journalisation complète et structurée
- ☒ Service *systemd* avec démarrage automatique
- ☒ Permissions fichiers correctement configurées
- ☒ Tests de détection validés

## Support et maintenance

### Mise à jour des règles :

- **Fréquence** → Quotidienne automatique via *suricata-update*
- **Source** → *Emerging Threats Open* (gratuite)
- Possibilité d'upgrade vers ET Pro pour couverture étendue

### Maintenance système :

- Rotation automatique des logs
- Nettoyage périodique du cache
- Surveillance des performances
- Vérification régulière de la configuration



**Documentation de référence :**

- **Documentation officielle Suricata** → <https://suricata.io/docs/>
- **Wiki Emerging Threats** → <https://doc.emergingthreats.net/>
- **Documentation Wazuh** → <https://documentation.wazuh.com/>
- **Forum communautaire** → <https://forum.suricata.io/>

